*Document number: P3015R0*
*Date: 2023-10-13*
*Audience: Library Evolution Working Group*
*Reply-to: Howard E. Hinnant*

# Rebuttal to Additional format specifiers for `time_point`

## Contents



## Introduction

This paper is written to express strong opposition to changing the meaning of `%S` as proposed in P2945R0. To be perfectly clear, this paper expresses no objection to adding syntax to specify precision to `%S` as proposed in the Less-Preferred Proposal Wording. This would not break any code. The only objection is to changing the meaning of `%S` from representing seconds exactly, to representing only the integral part of seconds as proposed in the Preferred Proposal Wording.

## Having a standard specify run-time changes to existing code is Bad™

The proper way to change existing behavior in a standard is to deprecate the existing behavior, and provide new syntax for the new behavior. Let both behaviors coexist for a few cycles, then remove the deprecated specification.

Doing otherwise risks introducing run-time errors to existing code by simply upgrading to a new version of C++. Such run-time errors have the potential for raising safety and security issues. It also undermines the important claims about C++ being stable and standards being backwards compatible.

## The `%S` flag as specified in the `<chrono>` library is existing code

It has been shipping in MSVC for over a year now.

It has been implemented in gcc 14.0.0.

`%S` has been documented by the standard, by books, by countless informal articles and postings, and by my date library.

`%S` has 7 years of positive field experience with this syntax and behavior in my date library.

## `%S` is a good design

The design of `%S` follows the principles first laid down by the C++11 version of `<chrono>`. `<chrono>` does not implicitly throw away information. This is true in conversion of units, and it remains true in formatting. All information is preserved unless truncating behavior is explicitly requested. And when it is explicitly requested, there exists options on which way to truncate: towards zero, towards negative infinity, towards positive infinity, or to nearest.

`<chrono>` was precision-neutral in C++11, and remains so a decade later in C++23. From N2661, written 2008-06-11:

> This paper proposes a solution that is *precision neutral*.

No single precision is promoted above others or given special privileges. It is not up to `<chrono>` to decide that one precision is better than another. That decision is up to the OS which talks directly to the hardware. And up to the client, who knows what precision is best for their application. `<chrono>` is just a middleman to make the syntax between the OS and the client portable.

Clients often change the OS's precision to their specification's precision at the point of input, such as wrapping a call to now:

```
auto
GetCurrentTime()
{
   using namespace std::chrono;
   return floor<milliseconds>(system_clock::now());
}

void
foo()
{
    auto tp = GetCurrentTime();
    // ...
    // maybe do some arithmetic or conversions to and from local time here
    // ...
    // Formatted with milliseconds precision as specified back in the implementation of GetCurrentTime()
    datafile << std::format("{:%FT%TZ}", tp);
}

void
bar()
{
    decltype(GetCurrentTime()) tp;
    // Same format string (minus the "{:}") to parse it back in
    datafile >> std::chrono::parse("%FT%TZ", tp);
}
```

And then traffic in those `time_points` throughout their library or application. There may be many points of both formatting and parsing involved throughout their application. And at each point they don't have to worry about synchronizing their parse and format statements with their specification's precision. They can just ask for the time: `"%F %T"` (etc.). And when their specification changes, there is no pain point. Formatting and parsing *by default* automatically adjust. Change `milliseconds` to `seconds` in `GetCurrentTime()` and things just work, but now at seconds-precision. Change `floor` to `ceil` or `round` to achieve alternative rounding modes from the OS-supplied precision.

`<chrono>` does not (and should not) carry all of this respect for the client around and then when the client gets ready to create a logging statement suddenly say: Oh, you probably want the precision a bunch of committee members decided upon. No, `<chrono>` should give the client the precision they have already asked for (as the default, of course there should be a way to change it).

Clients also value symmetry between the parsing and formatting strings so that they only have to learn one micro-language and not two. During parsing `%S` takes its precision from the type being parsed, and reads up to that amount of precision from the input stream. A piece-wise redesign of `<chrono>` without an in-depth knowledge of the entire library is foolhardy at best. And in this case is also disruptive and dangerous.

P2945R0 questions why the current spec offers fractional seconds, but not fractional minutes or hours. The answer[1] dates back to Egyptians who divided the day into 12 periods around B.C. 1500. At that time no one cared enough about fractional hours to invent a system for them. Hours were not divided into minutes until the first mechanical clocks that displayed minutes appeared near the end of the 16th century. Minutes were defined as $^1/_{60}$ of an hour based on the sexagesimal system developed by the Sumerians around 2000 B.C. This was used many centuries later first to divide circles, and later to divide the round mechanical clock faces. Minutes were subsequently divided using the sexagesimal system into seconds. In the 1500s seconds were not displayed on the clock face but rather measured by swings of the pendulum.

It was not until the late $19^{th}$ or early $20^{th}$ century that technology advanced enough for people to even consider dividing seconds. And at that time, civilization chose to use the base 10 system which had become well established. And this is why hours are divided by minutes, minutes by seconds, and seconds by powers of 10. It is simply history.

Nevertheless, if someone really wants fractional hours, that is certainly doable:

```
duration<double, hours::period> d = ...
cout << format("{:.4}, d) << '\n';
```

Finally `%S` does not need to have identical functionality to other languages as argued by P2945R0. There are many ways in which `<chrono>` differs from other date time libraries. These differences are what makes the `<chrono>` solution superior, in safety, functionality and performance.

# Acknowledgments

Thank you to Tomasz Kamiński, Stephan T. Lavavej, Bjarne Stroustrup, Alan Talbot, David Vandevoorde, and Ville Voutilainen for the corrections, suggestions and comments. This paper improved from its original draft because of your generously donated time and attention.

## References

[1] Why is a minute divided into 60 seconds, an hour into 60 minutes, yet there are only 24 hours in a day?